

## Programování 2

---

# 13. cvičení, 13-05-2026

---

## Farní oznamy

---

1. Tento text a kódy ke cvičení najdete v repozitáři cvičení na <https://github.com/PKvasnick/Programovani-2>.
2. Toto je **poslední** cvičení v tomto semestru, příští týden 19. května si napíšeme zápočtový test.
  - Přijdete na cvičení v obvyklém termínu 17:20.
  - Dostanete jedinou programovací úlohu, kterou vyřešíte přímo na cvičení ve vymezeném čase cca. 75 minut.
  - Řešení nahrajete do ReCodExu a tam najdete i hodnocení.
  - Následující den najdete v repozitáři řešení.

### 3. Zápočtový program

Pořád mi chybí tři témata zápočtového programu.

### 4. Domácí úkoly

Další domácí úkoly už v tomto semestru nedostanete. Momentálně je celkový počet bodů 200 a na zápočet potřebujete 70% bodů, tedy 140. Pokud dobře vidím, máte všichni splněno.

K úkolům na tento týden promluví jenom stručně.

---

### Dnešní program:

- Domácí úkoly
- Stabilní párování
- Dynamické programování:
  - Řezání tyče
  - Baťoh

---

## Domácí úkoly

---

S těmito úkoly jste si poradili dobře a tak je jenom proletíme.

## Lidské kruhy

Toto je vlastně lehký problém, protože stačí kráčet po grafu a sledovat, jestli jsme se nedostali k vrcholu, který jsme už viděli. Navíc prohledáváme lineární struktury, takže nikde nenarazíme na komplikaci.

```
# Count cycles in a permutation
from collections import deque

def read_from_console() -> list[int]:
    numbers = [int(s) for s in input().strip().split()]
    return numbers

def detect_cycles(permutation: list[int]) -> dict[int, deque[int]]:
    cycles = dict()
    done = [False] * len(permutation) # Nechceme prohledávat
    for pos0 in range(len(permutation)):
        if done[pos0]: # už jsme viděli
            continue
        k = pos0
        cycles[pos0] = deque() # deque, protože přidáváme postupně
        while not done[k]:
            done[k] = True
            cycles[pos0].append(k)
            k = permutation[k]
    return cycles

def main():
    permutation = read_from_console()
    result = detect_cycles(permutation)
    for cycle in result.values():
        print(*cycle)

if __name__ == "__main__":
    main()
```

**Otázka:** Proč u této úlohy nechceme použít Union Find (faktorovou množinu)?

Ne že by to nešlo. Ale u lineárních řetězců vlastně máme faktorové množiny od začátku a jenom hledáme, které to jsou.

## Bipartitní graf

Máme rozhodnout, jestli lze omalovat vrcholy grafu dvěma barvami tak, že každá hrana má vrcholy různé barvy?

Tato úloha má jednoduché řešení - prohledáme graf do hloubky nebo do šířky a obarvujeme vrcholy. Pokud jsme nenarazili na konflikt a omalovali jsme všechny vrcholy, pak je graf bipartitní.

Toto je typické řešení, které není ničím výjimečné.

```
from collections import defaultdict

def red_green():
    n_vertices = int(input())
    n_roads = int(input())
    graph = defaultdict(list)
    colors = [-1] * (n_vertices + 1)
    for _ in range(n_roads):
        start, end = [int(s) for s in input().split()]
        graph[start].append(end)
        graph[end].append(start)

    stack = [1] # DFS
    colors[1] = 1
    while stack:
        node = stack.pop()
        color = colors[node]
        for neighbour in graph[node]:
            if colors[neighbour] == -1: # Nevideli
                colors[neighbour] = not color
                stack.append(neighbour)
            elif colors[neighbour] == color: # Tady koncime.
                print("Nelze")
                return
    # Videli jsme vsechno?
    if -1 in colors[1:]:
        print("Nelze")
        return

    print(*[i for i in range(1, n_vertices + 1) if colors[i] == 1])
    print(*[i for i in range(1, n_vertices + 1) if colors[i] == 0])
    return

if __name__ == "__main__":
    red_green()
```

Stejná otázka jako u předchozí úlohy: proč nepoužíváme Union Find?

Protože to příliš nedává smysl a bylo by to podstatně složitější a pořád by to nedávalo smysl.

Bipartitní grafy jsou mimořádně užitečné a tak si hned uděláme malou ukázkou.

---

## Stabilní párování

Toto je problém optimálního párování a prototyp - stable marriage problem - je zábavný a džendrově nekorektní:

Máme  $L$  žen a  $R$  mužů. Každá žena má všechny muže setříděné podle preference, a také každý muž má ženy setříděné podle pořadí preference. Z těchto žen a mužů chceme vytvořit páry tak, aby párování bylo **stabilní** v tomto smyslu:

Pokud některá žena a muž nejsou vzájemně přiřazeni, pak nejméně jeden z nich preferuje stávajícího partnera.

Pokud existuje žena a muž, kteří se vzájemně preferují víc než stávající partnery, pak párování není stabilní a takovou dvojici nazýváme **blokující pár**. Tato dvojice bude totiž mít tendenci zrušit své stávající svazky a vytvořit nový pár. Stabilní párování tudíž nemá blokující pár.

Mějme tedy 4 muže a 4 ženy a jejich preference:

Wanda: Brent, Hank, Oscar, Davis

Emma: Davis, Hank, Oscar, Brent

Lacey: Brent, Davis, Hank, Oscar

Karen: Brent, Hank, Davis, Oscar

Oscar: Wanda, Karen, Lacey, Emma

Davis: Wanda, Lacey, Karen, Emma

Brent: Lacey, Karen, Wanda, Emma

Hank: Lacey, Wanda, Emma, Karen

Jedno stabilní párování je

Lacey a Brent (1 / 1)

Wanda a Hank (2 / 2)

Karen a Davis (3 / 3)

Emma a Oscar (3 / 4)

Stabilní párování tedy vůbec neznamená, že jsou všichni šťastní. Znamená jenom, že si žádná dvojice nemůže jednoduše polepšit.

Všimneme si, že toto párování neobsahuje blokující pár. Třeba Karen by dala přednost Brentovi a Hankovi před Davisem, ale Brent preferuje Lacey víc než Karen a Hank preferuje kohokoli víc než Karen, takže si Karen nepomůže.

Toto stabilní párování je unikátní, ale obecně to neplatí, pro daný set mužů, žen a jejich preferencí může existovat několik stabilních párování. Dá se ale ukázat, že stabilní párování vždy existuje.

Pro malé problémy není těžké najít řešení hrubou silou: Vygenerovat všechna možná párování (např. tak, že mužům postupně přiřadíme každou permutaci žen), a zjistíme, která permutace vede k nejvyšší spokojenosti (měřené součtem vzájemných preferencí všech výsledných párů) a u optimálního párování zkontrolujeme, že je stabilní.

Takovéto řešení má ale exponenciální složitost (pocházející od počtu permutací, které musíme prozkoumat). My si ukážeme si jednoduchý algoritmus s polynomiální složitostí:

**Gale-Shapleyův algoritmus** neboli algoritmus odloženého přijetí

- Každý účastník je svobodný nebo zadaný. Všichni účastníci začínají jako svobodní.
- Pár vznikne, když nezadaná žena navrhne svazek mužů.
- Když svobodný muž vytvoří pár, bude zadaný napořád, i když ne nutně se stejnou ženou.
- Když zadaný muž dostane návrh od ženy, kterou preferuje víc než stávající partnerku, vytvoří pár s

novou partnerkou a předchozí partnerka se stane nezadanou.

- Každá žena navrhuje svazek mužům postupně v pořadí podle preference, až vytvoří trvalý pár.
- Když jsou všichni zadaní, algoritmus končí a toto párování je stabilní.

#### Gale-Shapleyův algoritmus

1. Označíme všechny ženy a muže jako nezadané.
2. Dokud je nějaká žena  $Z$  svobodná:
  - Nechť je  $M$  nejvíc preferovaný muž v seznamu preferencí ženy  $Z$ , kterému ještě nenavrhla svazek
  - Pokud je muž  $M$  svobodný,  $Z$  a  $M$  vytvoří pár (a stanou se zadanými)
  - Pokud je muž  $M$  zadaný, ale ženu  $Z$  preferuje před stávající partnerkou  $Z'$ ,
    - .  $M$  zruší svazek se ženou  $Z'$ , která se stává svobodnou
    - .  $M$  a  $Z$  vytvoří pár a stanou se zadanými.
  - Jinak  $M$  odmítne  $Z$  a zůstává ve stávajícím páru.
3. Vrací seznam stabilních párů.

Gale-Shapleyův algoritmus vždy skončí a vrací stabilní párování. Složitost algoritmu je  $O(n^2)$  v čase i prostoru.

Zkusme si to napsat v Pythonu:

```
from collections import deque

def gale_shapley(men_preferences, women_preferences):
    n = len(women_preferences)
    free_women = deque(women_preferences.keys()) # All women start out free
    engaged_pairs = {} # Engaged pairs woman -> man
    women_proposals = {woman: [] for woman in women_preferences} # Track proposals

    # While there are free women who haven't proposed to all men
    while free_women:
        woman = free_women.popleft() # Grab the first free woman
        woman_pref = women_preferences[woman]
        for man in woman_pref:
            if man not in women_proposals[woman]:
                women_proposals[woman].append(man) # Record the proposal
                # If the man is free, engage them
                if man not in engaged_pairs.values():
                    engaged_pairs[woman] = man
                    # free_women.remove(woman)
                    break
            else:
                # If the man is already engaged, check if he prefers the new woman
                current_woman = next(
                    w for w, m in engaged_pairs.items() if m == man
                )
                man_pref = men_preferences[man]
                if man_pref.index(woman) < man_pref.index(current_woman):
                    # Engage the new woman and free the current one
                    engaged_pairs[woman] = man
                    # free_women.remove(woman)
                    free_women.append(current_woman)
```

```

        del engaged_pairs[current_woman]
        break

    return engaged_pairs

def main():
    # Example usage
    men_preferences = {
        "Oscar": ["Wanda", "Karen", "Lacey", "Emma"],
        "Davis": ["Wanda", "Lacey", "Karen", "Emma"],
        "Brent": ["Lacey", "Karen", "Wanda", "Emma"],
        "Hank": ["Lacey", "Wanda", "Emma", "Karen"],
    }

    women_preferences = {
        "Wanda": ["Brent", "Hank", "Oscar", "Davis"],
        "Emma": ["Davis", "Hank", "Oscar", "Brent"],
        "Lacey": ["Brent", "Davis", "Hank", "Oscar"],
        "Karen": ["Brent", "Hank", "Davis", "Oscar"],
    }

    result = gale_shapley(men_preferences, women_preferences)
    print(result)

if __name__ == "__main__":
    main()

```

Všimněme si, že toto je "women-first" verze Gale-Shapleyova algoritmu. Můžeme také vytvořit "men-first" verzi, ale předělat kód je docela otrava . Tady je:

```

from collections import deque

def gale_shapley(men_preferences, women_preferences):
    n = len(men_preferences)
    free_men = deque(men_preferences.keys()) # All men start out free
    engaged_pairs = {} # Engaged pairs man -> woman
    men_proposals = {man: [] for man in men_preferences} # Track proposals

    # While there are free men who haven't proposed to all women
    while free_men:
        man = free_men.popleft() # Grab the first free man
        man_pref = men_preferences[man]
        for woman in man_pref:
            if woman not in men_proposals[man]:
                men_proposals[man].append(woman) # Record the proposal
                # If the woman is free, engage them
                if woman not in engaged_pairs.values():
                    engaged_pairs[man] = woman

```

```

        break
    else:
        # If the woman is already engaged, check if she prefers the new man
        current_man = next(
            m for m, w in engaged_pairs.items() if w == woman
        )
        woman_pref = women_preferences[woman]
        if woman_pref.index(man) < woman_pref.index(current_man):
            # Engage the new man and free the current one
            engaged_pairs[man] = woman
            free_men.append(current_man)
            del engaged_pairs[current_man]
            break

    return engaged_pairs

def main():
    # Example usage
    men_preferences = {
        "Oscar": ["Wanda", "Karen", "Lacey", "Emma"],
        "Davis": ["Wanda", "Lacey", "Karen", "Emma"],
        "Brent": ["Lacey", "Karen", "Wanda", "Emma"],
        "Hank": ["Lacey", "Wanda", "Emma", "Karen"],
    }

    women_preferences = {
        "Wanda": ["Brent", "Hank", "Oscar", "Davis"],
        "Emma": ["Davis", "Hank", "Oscar", "Brent"],
        "Lacey": ["Brent", "Davis", "Hank", "Oscar"],
        "Karen": ["Brent", "Hank", "Davis", "Oscar"],
    }

    result = gale_shapley(men_preferences, women_preferences)
    print(result)

if __name__ == "__main__":
    main()

```

Dá se lehko ověřit, že obě verze dávají pro daná data stejné párování:

```
{'Brent': 'Lacey', 'Hank': 'Wanda', 'Davis': 'Karen', 'Oscar': 'Emma'}
```

Všimněme si, že to je docela mizerný kód, například hledá ve slovníku podle hodnot, ale ten slovník se používá také správně, takže to nejde úplně lehko vyřešit. Už tento kód obsahuje hodně datových struktur, a vyspělejší kódy se příliš nehodí pro rychlé seznámení.

Šlo by nějak vyřešit přepínání mezi "women-first" a "men-first" verzí tak, abychom nemuseli duplikovat kód?

## Pythonská vložka

Ale protože jsme měli v posledních cvičeních málo Pythonu, tady je třída implementující obousměrný slovník:

```
# Source - https://stackoverflow.com/a/21894086
# Posted by Basj, modified by community. See post 'Timeline' for change history
# Retrieved 2026-05-12, License - CC BY-SA 4.0

class bidict(dict):
    def __init__(self, *args, **kwargs):
        super(bidict, self).__init__(*args, **kwargs)
        self.inverse = {}
        for key, value in self.items():
            self.inverse.setdefault(value, []).append(key)

    def __setitem__(self, key, value):
        if key in self:
            self.inverse[self[key]].remove(key)
        super(bidict, self).__setitem__(key, value)
        self.inverse.setdefault(value, []).append(key)

    def __delitem__(self, key):
        self.inverse.setdefault(self[key], []).remove(key)
        if self[key] in self.inverse and not self.inverse[self[key]]:
            del self.inverse[self[key]]
        super(bidict, self).__delitem__(key)
```

Použití:

```
bd = bidict({'a': 1, 'b': 2})
print(bd)                # {'a': 1, 'b': 2}
print(bd.inverse)         # {1: ['a'], 2: ['b']}
bd['c'] = 1                # Now two keys have the same value (= 1)
print(bd)                 # {'a': 1, 'c': 1, 'b': 2}
print(bd.inverse)         # {1: ['a', 'c'], 2: ['b']}
del bd['c']
print(bd)                 # {'a': 1, 'b': 2}
print(bd.inverse)         # {1: ['a'], 2: ['b']}
del bd['a']
print(bd)                 # {'b': 2}
print(bd.inverse)         # {2: ['b']}
bd['b'] = 3
print(bd)                 # {'b': 3}
print(bd.inverse)         # {2: [], 3: ['b']}
```

Všimněme si, že zatímco přímý slovník je `dict[str : int]`, obrácený slovník je `dict[int: list[str]]`. Toto řešení výrazně zvyšuje praktickou použitelnost slovníku `bidict`.



### --- konec pythonské vložky ---

Příbuzný problém řešitelný Gale-Shapleyovým algoritmem je stabilní párování studentů na koleji. Tady nehledíme na pohlaví, jenom na vzájemné preference a data můžou vypadat nějak takto:

Wendy: Xenia, Yolanda, Zelda

Xenia: Wendy, Zelda, Yolanda

Yolanda: Wendy, Zelda, Xenia

Zelda: Xenia, Yolanda, Wendy

Jak se lehko přesvědčíte, stabilní párování je

Wendy a Xenia

Yolanda a Zelda

Na rozdíl od svatebního párování tady stabilní párování vůbec nemusí existovat.

Gale-Shapleyův algoritmus se úspěšně používá v řadě aplikací, např. při párování studentů a škol, uchazečů a zaměstnavatelů a pod. David Gale a Alvin Roth (který vynalezl podobný algoritmus dávno před Galem a Shapleym) dostali v roce 2012 Nobelovu cenu za ekonomii.

## Dynamické programování

### Příklad 1: Řezání tyče

Máme ocelovou tyč zadané délky a tabulku cen různě dlouhých kusů tyče. Máme tyč rozřezat tak, aby byl výnos z prodeje kousků co nejvyšší.

Tabulka cen může vypadat např. takto:

|       |  |   |   |   |   |    |    |    |    |
|-------|--|---|---|---|---|----|----|----|----|
| délka |  | 1 | 2 | 3 | 4 | 5  | 6  | 7  | 8  |
| cena  |  | 1 | 5 | 8 | 9 | 10 | 17 | 17 | 20 |

Když označíme  $x_i$  počet odřezků délky  $i$  a  $c_i$  jejich cenu, můžeme řešení formulovat jako optimalizační problém:

$$\max \sum_i x_i c_i$$

za podmínek

$$\sum_i i x_i \leq L$$
$$x_i \geq 0, i = 1, 2, \dots$$

Tuto úlohu lze vyřešit také hrubou silou pomocí rekurze:

```
řezy(cena, L) = max(cena[i] + řezy(cena, n-i-1)) pro všechny i z {0, 1 .. n-1}
```

V Pythonu to bude vypadat takto:

```
def cut_rod_naive(price, n):
    if n <= 0:
        return 0
    max_val = float("-inf")
    for i in range(n):
        max_val = max(max_val, price[i] + cut_rod_naive(price, n - i - 1))
    return max_val

def main():
    price = [1, 5, 8, 9, 10, 17, 17, 20, 24, 30]
    n = len(price)
    print(f"Maximum obtainable value is {cut_rod_naive(price, n)}")

if __name__ == "__main__":
    main()
```

To sice funguje, ale složitost algoritmu je katastrofální -  $O(2^L)$ . Složitosti můžeme výrazně napomocť memoizací, tedy kešováním vyřešených podproblémů. Toto se nazývá "top-down" dynamické programování, a nejdeme se jím zabývat, protože jsme podobné postupy používali při řešení problémů s rekurzí.

My ale nalezneme řešení jiným způsobem, a to "zdola nahoru" (bottom up) tak, že každý subproblém budeme automaticky řešit pouze jednou, protože si jeho řešení ihned uložíme. Přitom začneme řešením "malých" problémů a pokračujeme k větším, takže při řešení většího problému už máme k dispozici řešení všech menších problémů.

Ukládací prostor je pak cena, kterou platíme za to, že namísto algoritmu s exponenciální složitostí máme algoritmus s polynomiální složitostí.

Co budeme dělat? Budeme postupně hledat řešení pro délky 0, 1, 2, ... L a ukládat si je do pole.

```
def cut_rod_bottom_up(price, n):
    val = [0 for x in range(n + 1)]

    for i in range(1, n + 1):
        max_val = -1
        for j in range(i):
            max_val = max(max_val, price[j] + val[i - j - 1])
        val[i] = max_val

    return val

def main():
    # Example usage
    price = [1, 5, 8, 9, 10, 17, 17, 20]
    n = len(price)
    print("Prices:")
    print("", *range(1, n + 1), sep="\t")
    print("", *price, sep="\t")
```

```

val = cut_rod_bottom_up(price, n)
print("Subproblems table:")
print(*range(n + 1), sep="\t")
print(*val, sep="\t")

print(f"Maximum obtainable value is {val[-1]}")

if __name__ == "__main__":
    main()

```

Takto vypadá tabulka cen a tabulka maximálních výnosů pro délky 1 - 10:

```

Prices:
  1  2  3  4  5  6  7  8
  1  5  8  9 10 17 17 20
Subproblems table:
0  1  2  3  4  5  6  7  8
0  1  5  8 10 13 17 18 22
Maximum obtainable value is 22

```

Teď ale vzniká otázka, jak zjistit "vítězný" řez nebo kombinaci řezů. Jedno řešení je zapamatovat si první řez pro každé řešení subproblému:

```

def cut_rod_bottom_up(price, n):
    val = [0] * (n + 1)
    first = [0] * (n + 1) # stores the first cut for each subproblem

    for i in range(1, n + 1):
        max_val = -1
        for j in range(i):
            max_val = max(max_val, price[j] + val[i - j - 1])
            if max_val == price[j] + val[i - j - 1]:
                first[i] = j + 1
        val[i] = max_val

    return val, first

def main():
    # Example usage
    price = [1, 5, 8, 9, 10, 17, 17, 20]
    n = len(price)
    print("Prices:")
    print("", *range(1, n + 1), sep="\t")
    print("", *price, sep="\t")
    val, first = cut_rod_bottom_up(price, n)
    print("Subproblems table:")
    print(*range(n + 1), sep="\t")
    print(*val, sep="\t")
    print("First cuts:")

```

```
print(*first, sep="\t")
print(f"Maximum obtainable value is {val[-1]}")
```

```
if __name__ == "__main__":
    main()
```

a z výsledné tabulky lehce identifikujeme, jaká je optimální kombinace řezů:

```
Prices:
    1  2  3  4  5  6  7  8
    1  5  8  9 10 17 17 20
Subproblems table:
0  1  2  3  4  5  6  7  8
0  1  5  8 10 13 17 18 22
First cuts:
0  1  2  3  2  3  6  6  6
Maximum obtainable value is 22
```

První řez pro optimální hodnotu 22 pro délku 8 je 6. Co když ale máme víc řezů?

- 6 dává hodnotu 17, a optimální řez pro tyč délky 6 je žádný řez.
- Pro zbývajících kus délky 2 máme hodnotu 5 a optimální řez pro délku 2 je opět žádný řez.

Až teď je řešení kompletní.

## Příklad 2: Baťoh

$n$  položek  $s_i$ ,  $i = 1, 2, \dots, n$  s váhou  $w_i$  a cenou  $v_i$ . Najít podmnožinu položek tak, že

- jejich váha nepřesahuje  $W$
- jejich celková cena je největší ze všech podmnožin splňujících předchozí podmínku.

Naivní řešení je najít mezi přípustnými podmnožinami tu s největší celkovou cenou.

Lze dobře řešit rekurzivně:

Maximum pro  $n$ -tou hodnotu:

- Maximum pro váhu  $W$  a  $N-1$  položek (s vyloučením této hodnoty)
- Maximum pro váhu  $W - w[n]$  a  $N-1$  položek (se zařazením této hodnoty)

```
def knapSack(W, wt, val, n):

    # Base Case
    if n == 0 or W == 0:
        return 0

    # If weight of the nth item is more than knapsack of capacity W,
```

```
# then this item cannot be included in the optimal solution
if (wt[n-1] > w):
    return knapSack(w, wt, val, n-1)

# return the maximum of two cases:
# (1) nth item included
# (2) not included
else:
    return max(
        val[n-1] + knapSack(
            w-wt[n-1], wt, val, n-1),
        knapSack(w, wt, val, n-1))
```

Dynamické programování: budujeme tabulku: bereme maximální hodnotu z konfigurace bez zařazeného objektu a s ním.

| Weights (kg)                                                                        |                                                                                     |                                                                                     |                                                                                     | Knapsack capacities (kg) |     |     |     |     |     |      |       |      |      |       |  |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------|-----|-----|-----|-----|-----|------|-------|------|------|-------|--|
| 2                                                                                   | 1                                                                                   | 5                                                                                   | 3                                                                                   | 0                        | 1   | 2   | 3   | 4   | 5   | 6    | 7     | 8    | 9    | 10    |  |
|                                                                                     |                                                                                     |                                                                                     |                                                                                     | 0                        | 0   | 0   | 0   | 0   | 0   | 0    | 0     | 0    | 0    | 0     |  |
|    |                                                                                     |                                                                                     |                                                                                     | 0                        | 0   | 300 | 300 | 300 | 300 | 300  | 300   | 300  | 300  | 300   |  |
|  |  |                                                                                     |                                                                                     | 0                        | 200 | 300 | 500 | 500 | 500 | 500  | 500   | 500  | 500  | 500   |  |
|  |  |  |                                                                                     | 0                        | 200 | 300 | 500 | 500 | 500 | 600  | + 700 | 900  | 900  | 900   |  |
|  |  |  |  | 0                        | 200 | 300 | 500 | 700 | 800 | 1000 | 1000  | 1000 | 1100 | =1200 |  |
| 300                                                                                 | 200                                                                                 | 400                                                                                 | 500                                                                                 |                          |     |     |     |     |     |      |       |      |      |       |  |
| Values (\$)                                                                         |                                                                                     |                                                                                     |                                                                                     |                          |     |     |     |     |     |      |       |      |      |       |  |

```
# Program for 0-1 Knapsack problem
# Returns the maximum value that can
# be put in a knapsack of capacity w

def knapSack(W, wt, val, n):
    K = [[0 for x in range(W + 1)] for x in range(n + 1)]

    # Build table K[][] in bottom up manner
    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                K[i][w] = 0
            elif wt[i-1] <= w:
                K[i][w] = max(val[i-1]
                               + K[i-1][w-wt[i-1]],
                               K[i-1][w])
            else:
                K[i][w] = K[i-1][w]
```

```
    return K[n][w]

def main() -> None:
    profit = [60, 100, 120]
    weight = [10, 20, 30]
    w = 50
    print(knapsack(w, weight, profit, len(profit)))

if __name__ == '__main__':
    main()
```

## Domácí úkoly

---

Nejsou!